

SOLIDWORKS DimXpert API (.NET) — Examples (3/7): Get_DimXpert_Datum_Example_VB.NET Get_DimXpert_Sphere_Feature_Example_VB.NET

SOLIDWORKS API Documentation (converted from CHM)

Get DimXpert Datum Example (VB.NET)

This example shows how to build a part and get attributes for the DimXpert Datum annotation.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosfloxpress\ball valve\ball.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Add DimXpert datum icon on the DimXpert toolbar.  
' 4. Click the ball of the part.  
' 5. Click to place the datum annotation.  
' 6. Click the green check mark to accept the new datum feature.  
' 7. Observe the following DimXpert features on the DimXpertManager tab  
'    of the Management Panel: Sphere1.  
' 8. Open an Immediate window.  
' 9. Ensure that the SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click project in Project Explorer and  
'    click Add Reference > .NET tab).  
' 10. Ensure that the Microsoft Scripting Runtime library is loaded  
'    (right-click project in Project Explorer and click Add Reference >  
'    COM tab).  
'  
' Postconditions:  
' 1. Writes the output of this macro to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate Window.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
' -----  
  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Imports Scripting  
Partial Class SolidWorksMacro  
    Dim strs As New Collection  
    Dim boolstatus As Boolean  
    Dim dimXpertPart As DimXpertPart  
    Dim i As Double  
    Dim j As Double  
    Dim k As Double
```

```

Sub Main()
    Dim swModelDoc As IModelDoc2
    swModelDoc = swApp.ActiveDoc
    If swModelDoc Is Nothing Then
        Exit Sub
    End If
    Dim fso As New Scripting.FileSystemObject()
    Dim f As Scripting.File
    Dim textStr As Scripting.TextStream
    textStr = fso.CreateTextFile("c:\temp\dimXpertInfo.txt")
    f = fso.GetFile("c:\temp\dimXpertInfo.txt")
    'textStr = f.OpenAsTextStream(2, -2)
    If textStr Is Nothing Then
        Debug.Print("Error creating temp file.")
        Exit Sub
    End If
    Call log("Starting DimXpert log...", textStr)
    Call retrieve_info_text(swApp, textStr)
    textStr.Close()
End Sub

Private Sub log(ByVal text As String, ByVal textStr As Scripting.TextStream)
    Debug.Print(text)
    textStr.WriteLine(text)
End Sub

Private Sub retrieve_info_text(ByVal swapp As SldWorks, ByVal textStr As TextStream)
    Dim dimXpertMgr As SolidWorks.Interop.sldworks.DimXpertManager
    dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As DimXpertPart
    dimXpertPartObj = dimXpertMgr.DimXpertPart
    dimXpertPart = dimXpertPartObj
    Dim vAnnotations As Object
    vAnnotations = dimXpertPart.GetAnnotations()
    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)
    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        annotationTemp = vAnnotations(annotationIndex)
        Dim AnnotationDataText As Collection
        AnnotationDataText = AnnotationData(annotationTemp)
        Dim annotationTextIndex As Long

```

```

        For annotationTextIndex = 1 To AnnotationDataText.Count
            Call log(AnnotationDataText(annotationTextIndex), textStr)
        Next
    Next
End Sub

Public Function AnnotationData(ByVal annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long
    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertAnnotationType_e.swDimXpertDatum Then
        Call DatumData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Position Then
        Call PositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition Then
        Call CompositePositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry Then
        Call SymmetryData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity Then
        Call ConcentricityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile Then
        Call LineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
Then
        Call CompositeLineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile Then
        Call SurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
Then
        Call CompositeSurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Or annoType
= swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Or annoType =
swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        Call OrientationData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout Then
        Call TotalRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout Then
        Call CircularRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness Then
        Call FlatnessData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity Then
        Call CircularityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity Then
        Call CylindricityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness Then
        Call StraightnessData(annotation)

```

```

    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency Then
        Call TangencyData(annotation)
    Else ' any of the dimension tolerance types
        Call DimensionToleranceData(annotation)
    End If
    AnnotationData = strs
End Function

Private Sub Clear(ByVal strs As Collection)
    strs.Remove(strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub

Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Feature
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
    strs.Add("")
    strs.Add("Name: " + annotation.Name)
    annoType = annotationTypeNameFromObject(annotation)
    strs.Add("Type: " + annoType)
    strs.Add("Display Entity: " + DisplayEntity(annotation))
    modelObj = annotation.GetModelFeature()
    If Not (modelObj Is Nothing) Then
        strs.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")")
    End If
End Sub

Private Sub DatumData(ByVal annotation As DimXpertDatum)
    ' the datum letter
    strs.Add("")
    strs.Add("Datum Letter: " + annotation.Identifier)
End Sub

Private Sub PositionData(ByVal annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Position Tolerance Compartment:")
    ' the shape of the tolerance zone
    strs.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector if the tolerance zone is planar
    If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then

```

```

        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add(" Direction: " + FormatVector(I, J, K))
    End If
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositePositionData(ByVal annotation As DimXpertCompositePositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Composite Position Tolerance Compartment")
    ' the shape of the tolerance zone
    strs.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector when the zone is planar
    If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add(" Direction: " + FormatVector(I, J, K))
    End If
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier
    strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)
    ' the datum reference frame for the pattern location
    Call DatumsStr(annotation)
    ' the datum reference frame for the feature to feature location
    strs.Add("Composite datums:")
    strs.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub SymmetryData(ByVal annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double

```

```

    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Symmetry Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    strs.Add("")
    Call DatumsStr(annotation)
    ' the direction of the planar zone
    strs.Add("")
    boolstatus = annotation.GetZoneDirection(I, J, K)
    strs.Add("Planar Zone Direction: " + FormatVector(I, J, K))
End Sub

Private Sub ConcentricityData(ByVal annotation As DimXpertConcentricityTolerance)
    strs.Add("")
    strs.Add("Concentricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub TotalRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Total Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CircularRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circular Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub LineProfileData(ByVal annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")

```

```

    strs.Add("Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositeLineProfileData(ByVal annotation As
DimXpertCompositeLineProfileTolerance)
    strs.Add("")
    strs.Add("Composite Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(i, j, k)
    strs.Add("Planar Zone Vector: " + FormatVector(i, j, k))
    ' the datum reference frame
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub SurfaceProfileData(ByVal annotation As DimXpertSurfaceProfileTolerance)
    strs.Add("")
    strs.Add("Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositeSurfaceProfileData(ByVal annotation As
DimXpertCompositeSurfaceProfileTolerance)

```

```

    strs.Add("")
    strs.Add("Composite Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))
    ' the outer tolerance value
    strs.Add("  Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame for the location
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub OrientationData(ByVal annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    annoType = annotation.Type
    ' the type or orientation tolerance
    If annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        strs.Add("Orientation Type: Perpendicularity")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Then
        strs.Add("Orientation Type: Parallelism")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Then
        strs.Add("Orientation Type: Angularity")
    End If
    strs.Add("")
    strs.Add("Orientation Tolerance Compartment:")
    ' the shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Cylindrical
            strs.Add("  Zone Type: Cylindrical")
        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Planar
            strs.Add("  Zone Type: Planar")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
    End Select
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    'material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified

```

```

        boolstatus = annotation.GetProjectedZone(enabled, value)
        Call FormatProjectedZone(enabled, value)
        ' is tangent plane
        strs.Add("  IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))
        ' the datum reference frame
        Call DatumsStr(annotation)
End Sub

Private Sub FlatnessData(ByVal annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Flatness Tolerance Compartment:")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add("  Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add("    Per Unit Length: " + FormatDouble(length))
        strs.Add("    Per Unit Width: " + FormatDouble(width))
        strs.Add("    Per Unit Direction: " + FormatVector(I, J, K))
    End If
End Sub

Private Sub CircularityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circularity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub CylindricityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Cylindricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub StraightnessData(ByVal annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    strs.Add("")
    strs.Add("Straightness Tolerance Compartment")

```

```

'type or shape of the tolerance zone
Select Case annotation.ZoneType
    Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Cylindrical
        strs.Add(" Zone Type: Cylindrical")
    Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_PlanarMedian
        strs.Add(" Zone Type: Planar Median")
    Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Surface
        strs.Add(" Zone Type: Surface Straightness")
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add(" Zone Vector: " + FormatVector(I, J, K))
End Select
' the tolerance value
strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
'per unit length
boolstatus = annotation.GetPerUnitLength(enabled, dist)
strs.Add(" Per Unit Length: " + IIf(enabled, "True", "False"))
If enabled Then
    strs.Add(" Per Unit Length Distance: " + FormatDouble(dist))
End If
' the material condition modifier
strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
End Sub

Private Sub ProfileData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub SurfaceFinishData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Surface Finish Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub TangencyData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub DimensionToleranceData(ByVal annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long

```

```

Dim upper As Double, lower As Double
Dim plus As Double, minus As Double
Dim boolstatus As Boolean
annoType = annotation.Type
isAngleType = False
strs.Add("")
strs.Add("Dimension Tolerance Compartment")
If annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween Then
    Call DistanceBetweenData(annotation)
ElseIf annoType =
swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween Then
    Call CompositeDistanceBetweenData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore Then
    If annotation.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Depth Then
    If annotation.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
Then
    If annotation.ReferenceFeature Is Nothing Then
        strs.Add("Reference Feature: NULL")
    Else
        strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension Then
    Dim chamferAnno As IDimXpertChamferDimTol
    chamferAnno = annotation
    Select Case chamferAnno.ChamferType
        Case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_Angle
            strs.Add("Chamfer Dimension Type: Angle")
            isAngleType = True
        Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance1
            strs.Add("Chamfer Dimension Type: Distance 1")
        Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance2
            strs.Add("Chamfer Dimension Type: Distance 2")
    End Select
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween Then

```

```

        isAngleType = True
        ' the origin and tolerance feature
        strs.Add("Origin Feature: " + annotation.OriginFeature.Name)
        ' is supplement angle
        strs.Add("Supplement Angle: " + IIf(annotation.Supplement, "True", "False"))
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle Then
        isAngleType = True
        If annotation.ReferenceFeature Is Nothing Then
            strs.Add("Reference Feature: NULL")
        Else
            strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle Then
        isAngleType = True
    End If
    ' conversion for radians to degrees when dimension type is angle
    Dim dbl As Double
    If isAngleType Then
        dbl = 57.2957795130823
    Else
        dbl = 1.0#
    End If
    ' the nominal, and upper and lower limits of size of the dimension
    strs.Add("")
    strs.Add("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue *
dbl))
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
    strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
    ' the upper and lower tolerance value by type
    Select Case annotation.DimensionType
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance
            strs.Add("Dimension Type: Block Tolerance")
            ' block tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    strs.Add("Dimension Type: Block Tolerance No Nominal")
                    If isAngleType Then
                        strs.Add("Angular Block Tolerance")
                    Else
                        strs.Add("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))

```

```

        End If
        Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
            strsr.Add("Dimension Type: General Tolerance")
        End Select
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits
            strsr.Add("Dimension Type: Limits and Fits")
            ' limits and fits tolerance
        Case
            swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
                strsr.Add("Dimension Type: Limits and Fits No Nominal")
                strsr.Add("Limits and Fits: " + annotation.LimitsAndFitsCode)
                ' limit dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension
            strsr.Add("Dimension Type: Limit Dimension")
            boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
            strsr.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
            strsr.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance
            strsr.Add("Dimension Type: MAXTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance
            strsr.Add("Dimension Type: MINTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance
            strsr.Add("Dimension Type: NoTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension
            strsr.Add("Dimension Type: Plus Minus Dimension")
            ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            strsr.Add("Dimension Type: Plus Minus No Nominal")
            boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
            strsr.Add("Plus Tolerance: " + FormatDouble(plus * dbl))
            strsr.Add("Minus Tolerance: " + FormatDouble(minus * dbl))
        End Select
    End Sub

    Private Sub DistanceBetweenData(ByVal annotation As DimXpertDistanceBetweenDimTol)
        Dim feature As DimXpertFeature
        Dim featureFosUsage As Long
        Dim I As Double, J As Double, K As Double
        Dim boolstatus As Boolean
        feature = Nothing
        ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
        boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
        strsr.Add("")
        strsr.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        boolstatus = annotation.GetFeature(feature, featureFosUsage)
    End Sub

```

```

        strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        ' The direction vector
        boolstatus = annotation.GetDirectionVector(I, J, K)
        strs.Add("")
        strs.Add("Direction Vector: " + FormatVector(I, J, K))
    End Sub

    Private Sub CompositeDistanceBetweenData(ByVal annotation As
DimXpertCompositeDistanceBetweenDimTol)
        Dim feature As DimXpertFeature
        Dim featureFosUsage As Long
        Dim plus As Double, minus As Double
        Dim I As Double, J As Double, K As Double
        Dim boolstatus As Boolean
        feature = Nothing
        ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
        boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
        strs.Add("")
        strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        boolstatus = annotation.GetFeature(feature, featureFosUsage)
        strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        ' the pattern locating feature
        boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
        strs.Add("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))
        ' The direction vector
        boolstatus = annotation.GetDirectionVector(I, J, K)
        strs.Add("")
        strs.Add("Direction Vector: " + FormatVector(I, J, K))
        ' the upper and lower tolerance value for the pattern location by type
        Dim dimTol As IDimXpertDimensionTolerance
        dimTol = annotation
        Select Case dimTol.DimensionType
            ' plus and minus dimension
            Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
                dimTol.GetPlusAndMinusTolerance(plus, minus)
                strs.Add("Pattern Locating Dimension Type: Plus Minus No Nominal")
                strs.Add(" Plus Tolerance: " + FormatDouble(plus))
                strs.Add(" Minus Tolerance: " + FormatDouble(minus))
            ' block tolerance
            Case
                swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
                Dim blockTols As DimXpertBlockTolerances
                blockTols = dimXpertPart.GetBlockTolerances
                Select Case blockTols.Type
                    Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch

```

```

        str.Add("Pattern Locating Dimension Type: Block Tolerance No
Nominal")
        str.Add("  Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
        Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
            str.Add("Pattern Locating Dimension Type: General Tolerance")
        End Select
    End Select
    ' the upper and lower tolerance value for the feature to feature location by type
    Select Case annotation.DimensionTypeIntraFeature
        ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            str.Add("Feature Locating Dimension Type: Plus Minus No Nominal")
            boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
            str.Add("  Plus  Tolerance: " + FormatDouble(plus))
            str.Add("  Minus Tolerance: " + FormatDouble(minus))
            ' block tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    str.Add("Feature locating Dimension Type: Block Tolerance No
Nominal")
                    str.Add("  Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
                    Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                        str.Add("Feature locating Dimension Type: General Tolerance")
                    End Select
                End Select
            End Sub
        Private Sub DatumsStr(ByVal annotation As DimXpertTolerance)
            str.Add("")
            str.Add("Datums:")
            Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
            Call datumStr(annotation.GetSecondaryDatums(),
annotation.GetSecondaryDatumModifiers(), "  Secondary:")
            Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(),
"    Tertiary:")
        End Sub
        Private Sub datumStr(ByVal dats As Object, ByVal mods As Object, ByVal datumOrder As
String)
            Dim I As Long
            Dim str As String
            Dim mcm As Long
            If IsNothing(dats) Then

```

```

        Exit Sub
    End If
    str = ""
    For I = LBound(dats) To UBound(dats)
        mcm = mods(I)
        str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
    Next I
    If StrComp(str, "") > 0 Then
        strs.Add(datumOrder + str)
    Else
        strs.Add(datumOrder + " <none>")
    End If
End Sub

' returns a string containing the heith of the projected tolerance zone
Private Sub FormatProjectedZone(ByVal enabled As Boolean, ByVal height As Double)
    If enabled = True Then
        strs.Add(" Projected Zone: True")
        strs.Add(" Zone Height: " + FormatDouble(height))
    Else
        strs.Add(" Projected Zone: False")
    End If
End Sub

Private Function mcmStr(ByVal mcm As Long) As String
    Dim str As String
    str = ""
    Select Case mcm
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_LMC
            str = "LMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_MMC
            str = "MMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_NoMCM
            str = "NoMCM"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_RFS
            str = "RFS"
    End Select
    mcmStr = str
End Function

' returns a string containing the type of position tolerance zone used
Private Function PositionZoneType(ByVal typ As Long) As String
    Dim str As String
    Select Case typ
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_CylindricalPosition
            str = "Cylindrical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition
            str = "Planar"
    End Select
End Function

```

```

        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_SphericalPosition
            str = "Spherical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_Boundary
            str = "Boundary"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionArc
            str = "RadialPositionArc"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionPlanar
            str = "RadialPositionPlanar"
        Case Else
            str = "N/A"
    End Select
    PositionZoneType = str
End Function

' returns a string containing the names of the SW display entities
Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SolidWorks.Interop.sldworks.Annotation
    str = ""
    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    dispEnt = annotation.GetDisplayEntity()
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SolidWorks.Interop.sldworks.Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function

' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(ByVal value As Long) As String
    Dim str As String
    Select Case value
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center
            str = "Center"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide
            str = "Max"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide
            str = "Min"
        Case Else
            str = "N/A"
    End Select
    FosUsage = str
End Function

```

```

Private Function FormatVector(ByVal I As Double, ByVal J As Double, ByVal K As Double) As String
    Dim str As String
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
    FormatVector = str
End Function

Private Function FormatDouble(ByVal value As Double) As String
    Dim str As String
    str = Format(value, "##,##0.0000")
    FormatDouble = str
End Function

Private Function RadiansToDegrees(ByVal value As Double) As String
    Dim str As String
    str = Format((value * 57.2957795130823), "##,##0.00")
    RadiansToDegrees = str
End Function

Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String
    Select Case annoTypeIndex
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension
            annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween
            annotationTypeNameFromTypeNumber = "AngleBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle
            annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle
            annotationTypeNameFromTypeNumber = "ConeAngle Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter
            annotationTypeNameFromTypeNumber = "Diameter Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Length
            annotationTypeNameFromTypeNumber = "Length Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius
            annotationTypeNameFromTypeNumber = "Radius Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Width
            annotationTypeNameFromTypeNumber = "Width Dim"
    End Select
End Function

```

```

Case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween
    annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDatum
    annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout
    annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness
    annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity
    annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity
    annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness
    annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency
    annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
    annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)
End Select
End Function
Public swApp As SldWorks

```

End Class

Get_DimXpert_Display_Dimensions_and_Feature_Example_CSharp (Topic)

SOLIDWORKS API Help

Get DimXpert Display Dimensions and Feature (C#)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```
//-----  
// Preconditions:  
// 1. Add SolidWorks.Interop.swdimxpert.dll as a reference  
//    (in the Project Explorer, right-click References and click Add Reference).  
// 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.  
// 3. Click View > Toolbars > DimXpert.  
// 4. Click Auto Dimension Scheme on the DimXpert toolbar.  
// 5. Make sure Chamfer is selected in Feature Filters  
//    on the Auto Dimension Scheme PropertyManager page, then  
//    click the checkmark to close the page.  
//  
// Postconditions: Adds DimXpert display dimensions o the model.  
//  
// NOTE: Because this part document is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using SolidWorks.Interop.swdimxpert;  
using System;  
using System.Diagnostics;  
namespace GetDimXpertFeatureAnnotationCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swPart = default(ModelDoc2);  
            Annotation swAnnotation = default(Annotation);  
            DisplayDimension swDisplayDimension = default(DisplayDimension);  
  
            swPart = (ModelDoc2)swApp.ActiveDoc;  
            swAnnotation = (Annotation)swPart.GetFirstAnnotation2();  
            while (((swAnnotation != null)))  
            {  
                Debug.Print(" ");  
                Debug.Print("Annotation name = " + swAnnotation.GetName());  
                int AnnotationType = (int)swDimensionType_e.swDimensionTypeUnknown;  
                AnnotationType = swAnnotation.GetType();  
            }  
        }  
    }  
}
```

```

if (AnnotationType == (int)swAnnotationType_e.swDisplayDimension)
{
    Debug.Print(" Is a display dimension? True");
    swDisplayDimension = (DisplayDimension)swAnnotation.GetSpecificAnnotation();
    switch ((swDisplayDimension.Type2))
    {
        case (int)swDimensionType_e.swOrdinateDimension:
            Debug.Print(" Display dimension type = base ordinate and its
subordinates");
            break;
        case (int)swDimensionType_e.swLinearDimension:
            Debug.Print(" Display dimension type = linear");
            break;
        case (int)swDimensionType_e.swAngularDimension:
            Debug.Print(" Display dimension type = angular");
            break;
        case (int)swDimensionType_e.swArcLengthDimension:
            Debug.Print(" Display dimension type = arc length");
            break;
        case (int)swDimensionType_e.swRadialDimension:
            Debug.Print(" Display dimension type = radial");
            break;
        case (int)swDimensionType_e.swDiameterDimension:
            Debug.Print(" Display dimension type = diameter");
            break;
        case (int)swDimensionType_e.swHorOrdinateDimension:
            Debug.Print(" Display dimension type = horizontal ordinate");
            break;
        case (int)swDimensionType_e.swVertOrdinateDimension:
            Debug.Print(" Display dimension type = vertical ordinate");
            break;
        case (int)swDimensionType_e.swZAxisDimension:
            Debug.Print(" Display dimension type = z-axis");
            break;
        case (int)swDimensionType_e.swChamferDimension:
            Debug.Print(" Display dimension type = chamfer dimension");
            break;
        case (int)swDimensionType_e.swHorLinearDimension:
            Debug.Print(" Display dimension type = horizontal linear");
            break;
        case (int)swDimensionType_e.swVertLinearDimension:
            Debug.Print(" Display dimension type = vertical linear");
            break;
        case (int)swDimensionType_e.swScalarDimension:
            Debug.Print(" Display dimension type = scalar");

```

```

        break;
    default:
        Debug.Print(" Display dimension type = unknown");
        break;
    }
    Debug.Print(" Is a DimXpert display dimension? " +
(swDisplayDimension.IsDimXpert() == false ? "False" : "True"));
    if (swAnnotation.IsDimXpert())
    {
        DimXpertFeature DimXpertFeat = default(DimXpertFeature);
        string FeatName = null;
        DimXpertFeat = (DimXpertFeature)swAnnotation.GetDimXpertFeature();
        if ((DimXpertFeat != null))
        {
            FeatName = DimXpertFeat.Name;
            Debug.Print(" DimXpert feature name = " + FeatName);
            switch ((DimXpertFeat.Type))
            {
                case swDimXpertFeatureType_e.swDimXpertFeature_Plane:
                    Debug.Print(" DimXpert feature type = plane");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder:
                    Debug.Print(" DimXpert feature type = cylinder");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_Cone:
                    Debug.Print(" DimXpert feature type = cone");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_Extrude:
                    Debug.Print(" DimXpert feature type = extrude");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_Fillet:
                    Debug.Print(" DimXpert feature type = fillet");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer:
                    Debug.Print(" DimXpert feature type = chamfer");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole:
                    Debug.Print(" DimXpert feature type = compound hole");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth:
                    Debug.Print(" DimXpert feature type = compound width");
                    break;
                case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch:
                    Debug.Print(" DimXpert feature type = compound notch");
                    break;
            }
        }
    }
}

```

```

        case
swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D:
        Debug.Print(" DimXpert feature type = compound closed-slot
3D");

        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint:
        Debug.Print(" DimXpert feature type = intersect point");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine:
        Debug.Print(" DimXpert feature type = intersect line");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle:
        Debug.Print(" DimXpert feature type = intersect circle");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane:
        Debug.Print(" DimXpert feature type = intersect plane");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_Pattern:
        Debug.Print(" DimXpert feature type = pattern");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_Sphere:
        Debug.Print(" DimXpert feature type = sphere");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane:
        Debug.Print(" DimXpert feature type = best-fit plane");
        break;
    case swDimXpertFeatureType_e.swDimXpertFeature_Surface:
        Debug.Print(" DimXpert feature type = surface");
        break;
    default:
        Debug.Print(" DimXpert feature type = unknown");
        break;
    }
}
}
}
else
{
    Debug.Print(" Not a display dimension");
}
swAnnotation = (Annotation)swAnnotation.GetNext3();
}
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.

```

```
/// </summary>
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Get_DimXpert_Display_Dimensions_and_Feature_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Display Dimensions and Feature Example (VBA)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```
' -----  
' Preconditions:  
' 1. Ensure that the SOLIDWORKS DimXpert type library is  
'    loaded in Tools > References in the IDE.  
' 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt.  
' 3. Click View > Toolbars > DimXpert.  
' 4. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 5. Make sure Chamfer is selected in Feature Filters  
'    on the Auto Dimension Scheme PropertyManager page, then click the checkmark  
'    to close the page.  
'  
' Postconditions: Adds DimXpert display dimensions to the model.  
'  
' NOTE: Because this part document is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swPart As SldWorks.ModelDoc2  
Dim swAnnotation As SldWorks.Annotation  
Dim swDisplayDimension As SldWorks.DisplayDimension
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks  
    Set swPart = swApp.ActiveDoc  
    Set swAnnotation = swPart.GetFirstAnnotation2  
    Do While (Not (swAnnotation Is Nothing))  
        Debug.Print " "  
        Debug.Print "Annotation name = " & swAnnotation.GetName  
        If swAnnotationType_e.swDisplayDimension = swAnnotation.GetType Then  
            Debug.Print "  Is a display dimension? True"  
            Set swDisplayDimension = swAnnotation.GetSpecificAnnotation  
            Select Case (swDisplayDimension.Type2)  
                Case swDimensionType_e.swOrdinateDimension  
                    Debug.Print "    Display dimension type = base ordinate and its  
subordinates"  
                Case swDimensionType_e.swLinearDimension
```

```

        Debug.Print " Display dimension type = linear"
    Case swDimensionType_e.swAngularDimension
        Debug.Print " Display dimension type = angular"
    Case swDimensionType_e.swArcLengthDimension
        Debug.Print " Display dimension type = arc length"
    Case swDimensionType_e.swRadialDimension
        Debug.Print " Display dimension type = radial"
    Case swDimensionType_e.swDiameterDimension
        Debug.Print " Display dimension type = diameter"
    Case swDimensionType_e.swHorOrdinateDimension
        Debug.Print " Display dimension type = horizontal ordinate"
    Case swDimensionType_e.swVertOrdinateDimension
        Debug.Print " Display dimension type = vertical ordinate"
    Case swDimensionType_e.swZAxisDimension
        Debug.Print " Display dimension type = z-axis"
    Case swDimensionType_e.swChamferDimension
        Debug.Print " Display dimension type = chamfer dimension"
    Case swDimensionType_e.swHorLinearDimension
        Debug.Print " Display dimension type = horizontal linear"
    Case swDimensionType_e.swVertLinearDimension
        Debug.Print " Display dimension type = vertical linear"
    Case swDimensionType_e.swScalarDimension
        Debug.Print " Display dimension type = scalar"
    Case Else
        Debug.Print " Display dimension type = unknown"
End Select

```

```

        Debug.Print " Is a DimXpert display dimension? " &
IIf(swDisplayDimension.IsDimXpert = False, "False", "True")

```

```

If swAnnotation.IsDimXpert Then
    Dim DimXpertFeat As SwDimXpert.DimXpertFeature
    Dim FeatName As String

    Set DimXpertFeat = swAnnotation.GetDimXpertFeature()
    If Not DimXpertFeat Is Nothing Then
        FeatName = DimXpertFeat.Name
        Debug.Print " DimXpert feature name = " & FeatName

        Select Case (DimXpertFeat.Type)
            Case swDimXpertFeatureType_e.swDimXpertFeature_Plane
                Debug.Print " DimXpert feature type = plane"
            Case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder
                Debug.Print " DimXpert feature type = cylinder"
            Case swDimXpertFeatureType_e.swDimXpertFeature_Cone

```

```

        Debug.Print " DimXpert feature type = cone"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Extrude
        Debug.Print " DimXpert feature type =extrude"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Fillet
        Debug.Print " DimXpert feature type = fillet"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer
        Debug.Print " DimXpert feature type = chamfer"
    Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole
        Debug.Print " DimXpert feature type = compound hole"
    Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth
        Debug.Print " DimXpert feature type = compound width"
    Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch
        Debug.Print " DimXpert feature type = compound notch"
    Case
swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D
3D"
        Debug.Print " DimXpert feature type = compound closed-slot

    Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint
        Debug.Print " DimXpert feature type = intersect point"
    Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine
        Debug.Print " DimXpert feature type = intersect line"
    Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle
        Debug.Print " DimXpert feature type = intersect circle"
    Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane
        Debug.Print " DimXpert feature type = intersect plane"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Pattern
        Debug.Print " DimXpert feature type = pattern"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Sphere
        Debug.Print " DimXpert feature type = sphere"
    Case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane
        Debug.Print " DimXpert feature type = best-fit plane"
    Case swDimXpertFeatureType_e.swDimXpertFeature_Surface
        Debug.Print " DimXpert feature type = surface"
    Case Else
        Debug.Print " DimXpert feature type = unknown"
    End Select
End If
Else
    Debug.Print " Not a display dimension"
End If
Set swAnnotation = swAnnotation.GetNext3

Loop
End Sub

```

Get_DimXpert_Display_Dimensions_and_Feature_Example_VBNET (Topic)

SOLIDWORKS API Help

Get DimXpert Display Dimensions and Feature Example (VB.NET)

This example shows how to find out if an annotation is a DimXpert display dimension, and, if so, how to get its DimXpert feature.

```
' -----  
' Preconditions:  
' 1. Add SolidWorks.Interop.swdimxpert.dll as a reference  
'    (in the Project Explorer, right-click References and click Add Reference >  
'    .NET tab).  
' 2. Open public_documents\samples\tutorial\api\plate_tolstatus.sldprt  
' 3. Click View > Toolbars > DimXpert.  
' 4. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 5. Make sure Chamfer is selected in Feature Filters  
'    on the Auto Dimension Scheme PropertyManager page,  
'    then click the checkmark to close the page.  
'  
' Postconditions: Adds DimXpert display dimensions to the model.  
'  
' NOTE: Because this part document is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports SolidWorks.Interop.swdimxpert  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Public Sub Main()  
  
        Dim swPart As ModelDoc2  
        Dim swAnnotation As Annotation  
        Dim swDisplayDimension As DisplayDimension  
  
        swPart = swApp.ActiveDoc  
        swAnnotation = swPart.GetFirstAnnotation2  
        Do While (Not (swAnnotation Is Nothing))  
            Debug.Print(" ")  
            Debug.Print("Annotation name = " & swAnnotation.GetName)  
            If swAnnotationType_e.swDisplayDimension = swAnnotation.GetType Then  
                Debug.Print("  Is a display dimension? True")  
                swDisplayDimension = swAnnotation.GetSpecificAnnotation
```

```

Select Case (swDisplayDimension.Type2)
    Case swDimensionType_e.swOrdinateDimension
        Debug.Print("  Display dimension type = base ordinate and its
subordinates")

    Case swDimensionType_e.swLinearDimension
        Debug.Print("  Display dimension type = linear")
    Case swDimensionType_e.swAngularDimension
        Debug.Print("  Display dimension type = angular")
    Case swDimensionType_e.swArcLengthDimension
        Debug.Print("  Display dimension type = arc length")
    Case swDimensionType_e.swRadialDimension
        Debug.Print("  Display dimension type = radial")
    Case swDimensionType_e.swDiameterDimension
        Debug.Print("  Display dimension type = diameter")
    Case swDimensionType_e.swHorOrdinateDimension
        Debug.Print("  Display dimension type = horizontal ordinate")
    Case swDimensionType_e.swVertOrdinateDimension
        Debug.Print("  Display dimension type = vertical ordinate")
    Case swDimensionType_e.swZAxisDimension
        Debug.Print("  Display dimension type = z-axis")
    Case swDimensionType_e.swChamferDimension
        Debug.Print("  Display dimension type = chamfer dimension")
    Case swDimensionType_e.swHorLinearDimension
        Debug.Print("  Display dimension type = horizontal linear")
    Case swDimensionType_e.swVertLinearDimension
        Debug.Print("  Display dimension type = vertical linear")
    Case swDimensionType_e.swScalarDimension
        Debug.Print("  Display dimension type = scalar")
    Case Else
        Debug.Print("  Display dimension type = unknown")
End Select

Debug.Print("  Is a DimXpert display dimension? " &
IIf(swDisplayDimension.IsDimXpert = False, "False", "True"))

If swAnnotation.IsDimXpert Then
    Dim DimXpertFeat As DimXpertFeature
    Dim FeatName As String
    DimXpertFeat = swAnnotation.GetDimXpertFeature()
    If Not DimXpertFeat Is Nothing Then
        FeatName = DimXpertFeat.Name
        Debug.Print("  DimXpert feature name = " & FeatName)
        Select Case (DimXpertFeat.Type)
            Case swDimXpertFeatureType_e.swDimXpertFeature_Plane
                Debug.Print("  DimXpert feature type = plane")
            Case swDimXpertFeatureType_e.swDimXpertFeature_Cylinder
                Debug.Print("  DimXpert feature type = cylinder")
        End Select
    End If
End If

```

```

        Case swDimXpertFeatureType_e.swDimXpertFeature_Cone
            Debug.Print(" DimXpert feature type = cone")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Extrude
            Debug.Print(" DimXpert feature type = extrude")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Fillet
            Debug.Print(" DimXpert feature type = fillet")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Chamfer
            Debug.Print(" DimXpert feature type = chamfer")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole
            Debug.Print(" DimXpert feature type = compound hole")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth
            Debug.Print(" DimXpert feature type = compound width")
        Case swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch
            Debug.Print(" DimXpert feature type = compound notch")
        Case
            swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D
            Debug.Print(" DimXpert feature type = compound closed-slot
3D")

        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint
            Debug.Print(" DimXpert feature type = intersect point")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine
            Debug.Print(" DimXpert feature type = intersect line")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle
            Debug.Print(" DimXpert feature type = intersect circle")
        Case swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane
            Debug.Print(" DimXpert feature type = intersect plane")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Pattern
            Debug.Print(" DimXpert feature type = pattern")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Sphere
            Debug.Print(" DimXpert feature type = sphere")
        Case swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane
            Debug.Print(" DimXpert feature type = best-fit plane")
        Case swDimXpertFeatureType_e.swDimXpertFeature_Surface
            Debug.Print(" DimXpert feature type = surface")
        Case Else
            Debug.Print(" DimXpert feature type = unknown")
        End Select
    End If
End If
Else
    Debug.Print(" Not a display dimension")
End If
swAnnotation = swAnnotation.GetNext3
Loop
End Sub

```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Get_DimXpert_Extrude_Feature_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Extrude Feature Example (VBA)

This example shows how to build a part and get attributes for the DimXpert Extrude feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\plate_ads_plusminus.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create the Extrude Feature:  
'   a. Click Auto Dimension Scheme button on the DimXpert toolbar.  
'   b. Ensure that all feature filters are selected.  
'   c. Click the green check mark to accept the settings.  
' 4. Observe the following DimXpert features on the DimXpertManager tab:  
'   Hole Pattern1 and Pocket1.  
' 5. Open the Immediate window.  
' 6. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
'  
' Postconditions: Compare the output in the Immediate window with  
' the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfig As SldWorks.ConfigurationManager  
Dim swFeature As SldWorks.feature  
Dim swAnn As SldWorks.feature  
Dim swSchema As SldWorks.DimXpertManager  
Dim swDXPart As DimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String  
Dim msgStr3 As String  
Dim msgStr4 As String  
Dim n As Long  
Dim o As Long  
Dim p As Long  
Dim boolstatus As Boolean
```

```

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = "    " + "Model feature: "
        Set swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print msgStr + msgStr2
        End If

        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print msgStr + msgStr2
    
```

```

msgStr = "    " + "Number of applied features: "
msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print msgStr + msgStr2

```

```

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "        " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

```

```

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "        " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

```

```

Debug.Print "    "
Next

```

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
 ' which can return a general IDimXpertFeature or a more specific interface on the feature

' Get IDimXpertExtrudeFeature for the Pocket1 feature

```

Dim extrudeFeature As IDimXpertExtrudeFeature
Set extrudeFeature = swDXPart.GetFeature("Pocket1")
msgStr = extrudeFeature.Name + " is a DimXpert extrude feature"
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

```

' Get the bottom blends

```

Set feature = extrudeFeature.GetBottomBlends
If Not feature Is Nothing Then
    msgStr = "The bottom blends for the extrude is "
    msgStr2 = feature.Name
    Debug.Print msgStr + msgStr2
Else: Debug.Print "There are no bottom blends for this extrude."
End If

```

' Get the bottom feature

```

Set feature = extrudeFeature.GetBottomFeature

```

```

If Not feature Is Nothing Then
    msgStr = "The bottom feature for the extrude is "
    msgStr2 = feature.Name
    Debug.Print msgStr + msgStr2
End If

```

```

' Get the reference feature
Set feature = extrudeFeature.GetReferenceFeature
If Not feature Is Nothing Then
    msgStr = "The reference feature for the extrude is "
    msgStr2 = feature.Name
    Debug.Print msgStr + msgStr2
End If

```

```

' Get the sub-features
Dim featureCount As Integer
featureCount = extrudeFeature.GetSubFeatureCount
msgStr = "The number of sub-features of the extrude is "
msgStr2 = featureCount
Debug.Print msgStr + msgStr2

```

```

features = extrudeFeature.GetSubFeatures
For n = 0 To UBound(features)
    Set feature = features(n)
    Debug.Print " " + "Feature name: " + (feature.Name)

```

```

featureType = feature.Type
Call GetPatternType(featureType, msgStr2)

```

```

msgStr = "    Feature type is "

Debug.Print msgStr + msgStr2
Next

```

```

' Get whether the extrude is blind
boolstatus = extrudeFeature.Blind
msgStr = "The extrude is blind: "
msgStr2 = boolstatus
Debug.Print msgStr + msgStr2

```

```

End Sub

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    End If
End Sub

```

```
ElseIf (featureType = swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
ElseIf (featureType = swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub
```

Get DimXpert Extrude Feature Example (VB.NET)

This example shows how to build a part and get attributes for the DimXpert Extrude feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\plate_ads_plusminus.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create the Extrude Feature:  
'   a. Click Auto Dimension Scheme on the DimXpert toolbar.  
'   b. Ensure the Scope is set to All Features.  
'   c. Click the green check mark to accept the settings.  
' 4. Observe the following DimXpert features on the DimXpertManager tab:  
'   Hole Pattern1 and Pocket1.  
' 5. Open the Immediate window.  
' 6. Ensure that the SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click project in Project Explorer and  
'    click Add Reference > .NET tab).  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Partial Class SolidWorksMacro  
  
    Dim swModel As IModelDoc2  
    Dim swModelDocExt As IModelDocExtension  
    Dim swSelMgr As ISelectionMgr  
    Dim swConfig As IConfiguration  
    Dim swFeature As IFeature  
    Dim swAnn As IFeature  
    Dim swSchema As IDimXpertManager  
    Dim swDXPart As IDimXpertPart  
    Dim featureType As swDimXpertFeatureType_e  
    Dim features As Object  
    Dim appliedFeatures As Object  
    Dim appliedAnnotations As Object  
    Dim appliedAnnotation As IDimXpertAnnotation  
    Dim feature As IDimXpertFeature  
    Dim appliedFeature As IDimXpertFeature  
    Dim msgStr As String
```

```

Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "   " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "   " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "   " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then
            For o = 0 To UBound(appliedFeatures)

```

```

        appliedFeature = appliedFeatures(o)
        Debug.Print("    " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If
msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print("    ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
' Get IDimXpertExtrudeFeature for the Pocket1 feature
Dim extrudeFeature As IDimXpertExtrudeFeature
extrudeFeature = swDXPart.GetFeature("Pocket1")
msgStr = extrudeFeature.Name + " is a DimXpert extrude feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the bottom blends
feature = extrudeFeature.GetBottomBlends
If Not feature Is Nothing Then
    msgStr = "The bottom blends for the extrude is "
    msgStr2 = feature.Name
    Debug.Print(msgStr + msgStr2)
Else : Debug.Print("There are no bottom blends for this extrude.")
End If
' Get the bottom feature
feature = extrudeFeature.GetBottomFeature
If Not feature Is Nothing Then
    msgStr = "The bottom feature for the extrude is "
    msgStr2 = feature.Name
    Debug.Print(msgStr + msgStr2)
End If
' Get the reference feature
feature = extrudeFeature.GetReferenceFeature
If Not feature Is Nothing Then
    msgStr = "The reference feature for the extrude is "
    msgStr2 = feature.Name
    Debug.Print(msgStr + msgStr2)
End If
' Get the sub-features
Dim featureCount As Integer
featureCount = extrudeFeature.GetSubFeatureCount
msgStr = "The number of sub-features of the extrude is "

```

```

msgStr2 = featureCount
Debug.Print(msgStr + msgStr2)
features = extrudeFeature.GetSubFeatures
For n = 0 To UBound(features)
    feature = features(n)
    Debug.Print(" " + "Feature name: " + (feature.Name))
    featureType = feature.Type
    Call GetPatternType(featureType, msgStr2)
    msgStr = "    Feature type is "
    Debug.Print(msgStr + msgStr2)
Next
' Get whether the extrude is blind
boolstatus = extrudeFeature.Blind
msgStr = "The extrude is blind: "
msgStr2 = boolstatus
Debug.Print(msgStr + msgStr2)
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then

```

```
msgStr2 = "Bestfit Plane"
```

```
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
```

```
msgStr2 = "Surface"
```

```
End If
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Get_DimXpert_Feature2_Example_VB (Topic)

SOLIDWORKS API Help

Get More DimXpert Feature Examples (VBA)

This example shows how to build and get attributes for the following DimXpert features:

- * Hole
- * Notch

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilcover_with_dimensions.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Observe the following DimXpert features on the DimXpertManager tab:  
'   Simple Hole and, Notch1.  
' 4. Open the Immediate window.  
' 5. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
' 6. Step through this macro (F8).  
'  
' Postconditions: Compare the output in the Immediate window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As ModelDoc2  
Dim swModelDocExt As ModelDocExtension  
Dim swSelMgr As SelectionMgr  
Dim swConfig As Configuration  
Dim swFeature As feature  
Dim swAnn As feature  
Dim swSchema As DimXpertManager  
Dim swDXPart As DimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim holeType As swDimXpertCompoundHoleType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String  
Dim msgStr3 As String  
Dim msgStr4 As String  
Dim n As Long  
Dim o As Long  
Dim p As Long  
Dim boolstatus As Boolean
```

```

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = "    " + "Model feature: "
        Set swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeName2()
            Debug.Print msgStr + msgStr2
        End If

        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print msgStr + msgStr2
    
```

```

msgStr = "    " + "Number of applied features: "
msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print msgStr + msgStr2

```

```

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "    " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

```

```

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "    " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

```

```

Debug.Print "    "
Next

```

```

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

```

```

' Get IDimXpertCompoundHoleFeature for the Simple Hole2 feature

```

```

Dim holeFeature As IDimXpertCompoundHoleFeature
Set holeFeature = swDXPart.GetFeature("Simple Hole2")
msgStr = holeFeature.Name + " is a DimXpert feature"
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

```

```

' Get the bottom feature if one exists
Dim bottomFeature As IDimXpertFeature
Set bottomFeature = holeFeature.GetBottomFeature
If Not (bottomFeature Is Nothing) Then
    msgStr = "Bottom feature is "
    msgStr2 = bottomFeature.Name
    Debug.Print msgStr + msgStr2
End If

```

```

' Get the reference feature
Dim refFeature As IDimXpertFeature

```

```

Set refFeature = holeFeature.GetReferenceFeature
msgStr = "Reference feature for dimensioning is "
msgStr2 = refFeature.Name
Debug.Print msgStr + msgStr2

' Get the sub-feature count
Dim count As Integer
count = holeFeature.GetSubFeatureCount
msgStr = "Number of subfeatures is "
msgStr2 = count
Debug.Print msgStr + msgStr2

' Get the sub-features
Dim subfeatures As Variant
subfeatures = holeFeature.GetSubFeatures
For n = 0 To UBound(subfeatures)
    Set feature = subfeatures(n)
    Debug.Print " " + "Sub-feature name: " + (feature.Name)
Next

' Get whether the hole is blind
msgStr = "Hole feature is blind and not through: "
msgStr2 = holeFeature.Blind
Debug.Print msgStr + msgStr2

' Get the type of the hole
holeType = holeFeature.CompoundHoleType
msgStr = "Hole feature is type: "
Call GetHoleType(holeType, msgStr2)
Debug.Print msgStr + msgStr2

' Get IDimXpertCompoundNotchFeature for the Notch1 feature

Dim notchFeature As IDimXpertCompoundNotchFeature
Set notchFeature = swDXPart.GetFeature("Notch1")
msgStr = notchFeature.Name + " is a DimXpert feature"
    Debug.Print ""
    Debug.Print msgStr
Debug.Print ""

' Get the nominal notch coordinates

Dim width As Double
Dim length As Double
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double

```

```
Dim longitudeI As Double
Dim longitudeJ As Double
Dim longitudeK As Double
```

```
Debug.Print "Nominal notch of Notch1"
Debug.Print ""

boolstatus = notchFeature.GetNominalNotch(width, length, x, y, z, i, j, k, longitudeI, longitudeJ, longitudeK)
msgStr = "Width is "
msgStr2 = width
Debug.Print msgStr + msgStr2
msgStr = "Length is "
msgStr2 = length
Debug.Print msgStr + msgStr2
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
msgStr = "I-component of pierce vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "J-component of pierce vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "K-component of pierce vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
msgStr = "I-component of longitudinal unit vector is "
msgStr2 = longitudeI
Debug.Print msgStr + msgStr2
msgStr = "J-component of longitudinal unit vector is "
msgStr2 = longitudeJ
Debug.Print msgStr + msgStr2
msgStr = "K-component of longitudinal unit vector is "
msgStr2 = longitudeK
Debug.Print msgStr + msgStr2
Debug.Print ""
```

```
End Sub
```

```
Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
```

```

        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
        msgStr2 = "Bestfit plane"
    ElseIf (featureType = swDimXpertFeature_Surface) Then
        msgStr2 = "Surface"
    End If

End Sub

Public Sub GetHoleType(ByRef holeType, ByRef msgStr2)
    If (holeType = swDimXpertCompoundHoleType_Compound) Then
        msgStr2 = "Compound"
    ElseIf (holeType = swDimXpertCompoundHoleType_Counterbore) Then
        msgStr2 = "Counterbore"
    ElseIf (holeType = swDimXpertCompoundHoleType_Countersink) Then
        msgStr2 = "Countersink"
    ElseIf (holeType = swDimXpertCompoundHoleType_Simple) Then
        msgStr2 = "Simple"
    End If

End Sub

```

Get More DimXpert Feature Examples (VB.NET)

This example shows how to build and get attributes for the following DimXpert features:

- * Hole
- * Notch

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilcover_with_dimensions.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Observe the following DimXpert features on the DimXpertManager tab:  
'   Simple Hole2 and Notch1.  
' 4. Open the Immediate window.  
' 5. Ensure that the latest SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click the project in Project Explorer and  
'    click Add Reference > .NET tab)  
' 6. Step through this macro (F8).  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swdimxpert
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As IModelDoc2
```

```
    Dim swModelDocExt As IModelDocExtension
```

```
    Dim swSelMgr As ISelectionMgr
```

```
    Dim swConfig As IConfiguration
```

```
    Dim swFeature As IFeature
```

```
    Dim swAnn As IFeature
```

```
    Dim swSchema As IDimXpertManager
```

```
    Dim swDXPart As IDimXpertPart
```

```
    Dim featureType As swDimXpertFeatureType_e
```

```
    Dim holeType As swDimXpertCompoundHoleType_e
```

```
    Dim features As Object
```

```
    Dim appliedFeatures As Object
```

```
    Dim appliedAnnotations As Object
```

```
    Dim appliedAnnotation As IDimXpertAnnotation
```

```
    Dim feature As IDimXpertFeature
```

```
    Dim appliedFeature As IDimXpertFeature
```

```
    Dim msgStr As String
```

```
    Dim msgStr2 As String
```

```

Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "    " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeName2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "    " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then
            For o = 0 To UBound(appliedFeatures)
                appliedFeature = appliedFeatures(o)
            
```

```

        Debug.Print("    " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If
msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print(" ")
Next
' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
' Get IDimXpertCompoundHoleFeature for the Simple Hole2 feature
Dim holeFeature As IDimXpertCompoundHoleFeature
holeFeature = swDXPart.GetFeature("Simple Hole2")
msgStr = holeFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the bottom feature if one exists
Dim bottomFeature As IDimXpertFeature
bottomFeature = holeFeature.GetBottomFeature
If Not (bottomFeature Is Nothing) Then
    msgStr = "Bottom feature is "
    msgStr2 = bottomFeature.Name
    Debug.Print(msgStr + msgStr2)
End If
' Get the reference feature
Dim refFeature As IDimXpertFeature
refFeature = holeFeature.GetReferenceFeature
msgStr = "Reference feature for dimensioning is "
msgStr2 = refFeature.Name
Debug.Print(msgStr + msgStr2)
' Get the sub-feature count
Dim count As Integer
count = holeFeature.GetSubFeatureCount
msgStr = "Number of subfeatures is "
msgStr2 = count
Debug.Print(msgStr + msgStr2)
' Get whether the hole is blind
msgStr = "Hole feature is blind and not through: "
msgStr2 = holeFeature.Blind
Debug.Print(msgStr + msgStr2)
' Get the type of the hole
holeType = holeFeature.CompoundHoleType
msgStr = "Hole feature is type: "

```

```

Call GetHoleType(holeType, msgStr2)
Debug.Print(msgStr + msgStr2)
' Get IDimXpertCompoundNotchFeature for the Notch1 feature
Dim notchFeature As IDimXpertCompoundNotchFeature
notchFeature = swDXPart.GetFeature("Notch1")
msgStr = notchFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the nominal notch coordinates
Dim width As Double
Dim length As Double
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
Dim longitudeI As Double
Dim longitudeJ As Double
Dim longitudeK As Double
Debug.Print("Nominal notch of Notch1")
Debug.Print("")
boolstatus = notchFeature.GetNominalNotch(width, length, x, y, z, i, j, k, longitudeI, longitudeJ, longitudeK)
msgStr = "Width is "
msgStr2 = width
Debug.Print(msgStr + msgStr2)
msgStr = "Length is "
msgStr2 = length
Debug.Print(msgStr + msgStr2)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
msgStr = "I-component of unit vector is "
msgStr2 = i
Debug.Print(msgStr + msgStr2)
msgStr = "J-component of unit vector is "
msgStr2 = j
Debug.Print(msgStr + msgStr2)
msgStr = "K-component of unit vector is "
msgStr2 = k
Debug.Print(msgStr + msgStr2)
msgStr = "Longitudinal i-component of pierce unit vector is "
msgStr2 = longitudeI
Debug.Print(msgStr + msgStr2)

```

```

msgStr = "Longitudinal j-component of pierce unit vector is "
msgStr2 = longitudeJ
Debug.Print(msgStr + msgStr2)

msgStr = "Longitudinal k-component of pierce unit vector is "
msgStr2 = longitudeK
Debug.Print(msgStr + msgStr2)
Debug.Print("")

End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Extrude) Then
        msgStr2 = "Boss"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_BestfitPlane) Then
        msgStr2 = "Bestfit plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Surface) Then
        msgStr2 = "Surface"
    End If
End Sub

Public Sub GetHoleType(ByRef holeType As swDimXpertCompoundHoleType_e, ByRef msgStr2 As String)
    If (holeType = swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Compound) Then
        msgStr2 = "Compound"
    ElseIf (holeType = swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Counterbore) Then

```

```
        msgStr2 = "Counterbore"
    ElseIf (holeType = swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Countersink) Then
        msgStr2 = "Countersink"
    ElseIf (holeType = swDimXpertCompoundHoleType_e.swDimXpertCompoundHoleType_Simple) Then
        msgStr2 = "Simple"
    End If
End Sub

Public swApp As SldWorks
End Class
```

Get_DimXpert_Feature_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Feature Example (VBA)

This example builds a part to demonstrate usage of the DimXpert API for the following DimXpert features:

- * Cylinder
- * Fillet
- * Plane
- * Slot
- * Pattern

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilcover_plate.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 4. Ensure that all feature filters are selected.  
' 5. Click the green check mark to accept the settings.  
' 6. Observe the following DimXpert features on the DimXpertManager tab:  
'   Cylinder, Fillet, Plane, Slot, Slot Pattern.  
' 7. Open the Immediate window.  
' 8. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
' 9. Step through this macro (F8) and compare the output in the  
'   Immediate window with the features displayed in DimXpertManager.  
'  
' Postconditions: Compare the output in the Immediate window with the features  
' displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swConfig As SldWorks.Configuration  
Dim swFeature As SldWorks.feature  
Dim swAnn As SldWorks.feature  
Dim swSchema As SldWorks.DimXpertManager  
Dim swDXPart As DimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim features As Variant  
Dim appliedFeatures As Variant  
Dim appliedAnnotations As Variant  
Dim appliedAnnotation As DimXpertAnnotation  
Dim feature As DimXpertFeature  
Dim appliedFeature As DimXpertFeature  
Dim msgStr As String
```

```

Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        'Use IDimXpertFeature to get feature data
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = "   " + "Swift model: "

        'Use IFeature to get the Swift model corresponding to this geometric dimensioning and tolerance data
        Set swFeature = feature.GetModelFeature()

```

```

If Not (swFeature Is Nothing) Then
    msgStr2 = swFeature.GetTypeName2()
    Debug.Print msgStr + msgStr2
End If

```

```

msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
msgStr2 = feature.GetFaceCount
Debug.Print msgStr + msgStr2

```

```

msgStr = "    " + "Number of applied features: "
msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print msgStr + msgStr2

```

```

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "        " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

```

```

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "        " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

```

```

Debug.Print "    "
Next

```

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
 ' which can return a general IDimXpertFeature or a more specific interface on the feature

```

Dim cylinderFeature As IDimXpertCylinderFeature
Set cylinderFeature = swDXPart.GetFeature("Cylinder1")
msgStr = cylinderFeature.Name + " is a DimXpert feature"
Debug.Print ""
Debug.Print msgStr

```

' Get the nominal top plane coordinates for Cylinder1

```

Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double

```

```

Dim j As Double
Dim k As Double
boolstatus = cylinderFeature.GetNominalTopPlane(x, y, z, i, j, k)
msgStr = "X-coordinate of nominal top plane of " + cylinderFeature.Name + " is "
msgStr2 = x
Debug.Print msgStr + msgStr2

' Get the thread for the cylinder, if it exists

Dim isThreaded As Boolean
Dim threadDesignation As String
Dim threadDepth As Double
boolstatus = cylinderFeature.GetThread(isThreaded, threadDesignation, threadDepth)
msgStr = "The cylinder is threaded? "
msgStr2 = isThreaded
Debug.Print msgStr + msgStr2
Debug.Print ""

' Get the radius of Fillet1

Dim filletFeature As IDimXpertFilletFeature
Set filletFeature = swDXPart.GetFeature("Fillet1")
msgStr = filletFeature.Name + " is a DimXpert feature"
    Debug.Print ""
    Debug.Print msgStr
Dim radius As Double
radius = filletFeature.radius
msgStr = "The fillet radius is "
msgStr2 = radius
Debug.Print msgStr + msgStr2

Dim planeFeature As IDimXpertPlaneFeature
Set planeFeature = swDXPart.GetFeature("Plane1")
msgStr = planeFeature.Name + " is a DimXpert feature"
    Debug.Print ""
    Debug.Print msgStr

' Get the nominal top plane coordinates for Plane1

boolstatus = planeFeature.GetNominalPlane(x, y, z, i, j, k)
msgStr = "X-coordinate of nominal top plane of " + planeFeature.Name + " is "
msgStr2 = x
Debug.Print msgStr + msgStr2

' Get the nominal closed slot width for Slot1

Dim slotFeature As IDimXpertCompoundClosedSlot3DFeature
Set slotFeature = swDXPart.GetFeature("Slot1")
msgStr = slotFeature.Name + " is a DimXpert feature"
    Debug.Print ""
    Debug.Print msgStr

```

```

Dim width As Double
Dim length As Double
Dim longitudeI As Double
Dim longitudeJ As Double
Dim longitudeK As Double
boolstatus = slotFeature.GetNominalClosedSlot(width, length, x, y, z, i, j, k, longitudeI, longitudeJ, longitudeK)
msgStr = "Width of nominal closed slot is "
msgStr2 = width
Debug.Print msgStr + msgStr2

' Get the sub-features of Slot Pattern1

Dim patternFeature As IDimXpertPatternFeature
Set patternFeature = swDXPart.GetFeature("Slot Pattern1")
msgStr = patternFeature.Name + " is a DimXpert feature"
    Debug.Print ""
    Debug.Print msgStr

featureType = patternFeature.PatternType
Call GetPatternType(featureType, msgStr2)

Dim featureCount As Integer
featureCount = patternFeature.GetSubFeatureCount()
msgStr = "    Number of "
msgStr3 = featureCount
Debug.Print msgStr + msgStr2 + " is " + msgStr3

Dim subfeatures As Variant
subfeatures = patternFeature.GetSubFeatures()
Debug.Print "    Sub-features of Slot Pattern1:"
For n = 0 To UBound(subfeatures)
    ' Use IDimXpertCompoundClosedSlot3DFeature to get the two planes of the slot
    ' The names of the planes are inherited from the Name property of the parent interface, IDimXpertFeature
    Set slotFeature = subfeatures(n)
    Debug.Print "        " + slotFeature.Name
    Dim plane1 As IDimXpertPlaneFeature
    Dim plane2 As IDimXpertPlaneFeature
    boolstatus = slotFeature.GetPlaneFeatures(plane1, plane2)
    Debug.Print "            " + plane1.Name
    Debug.Print "            " + plane2.Name
Next
End Sub

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    End If
End Sub

```

```
ElseIf (featureType = swDimXpertFeature_Fillet) Then
    msgStr2 = "Fillet"
ElseIf (featureType = swDimXpertFeature_Chamfer) Then
    msgStr2 = "Chamfer"
ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
    msgStr2 = "CompoundHole"
ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
    msgStr2 = "CompoundWidth"
ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
    msgStr2 = "CompoundNotch"
ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
    msgStr2 = "CompoundClosedSlot3D"
ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
    msgStr2 = "IntersectPoint"
ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
    msgStr2 = "IntersectLine"
ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
    msgStr2 = "IntersectCircle"
ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
    msgStr2 = "IntersectPlane"
ElseIf (featureType = swDimXpertFeature_Pattern) Then
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub
```

Get DimXpert Feature Example (VB.NET)

This example builds a part to demonstrate usage of the DimXpert API for the following DimXpert features:

- * Cylinder
- * Fillet
- * Plane
- * Slot
- * Pattern

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilcover_plate.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 4. Ensure that all of the feature filters are selected.  
' 5. Click the green check mark to accept the settings.  
' 6. Observe the following DimXpert feature types on the DimXpertManager tab:  
'   Cylinder, Fillet, Plane, Slot, Pattern.  
' 7. Open the Immediate window.  
' 8. Ensure that the latest SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click project in Project Explorer and click Add Reference >  
'    .NET tab).  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System.Diagnostics  
Imports System  
  
Partial Class SolidWorksMacro  
    Dim swModel As IModelDoc2  
    Dim swModelDocExt As IModelDocExtension  
    Dim swSelMgr As ISelectionMgr  
    Dim swConfig As IConfiguration  
    Dim swFeature As IFeature  
    Dim swAnn As IFeature  
    Dim swSchema As IDimXpertManager  
    Dim swDXPart As IDimXpertPart  
    Dim featureType As swDimXpertFeatureType_e  
    Dim features As Object  
    Dim appliedFeatures As Object  
    Dim appliedAnnotations As Object  
    Dim appliedAnnotation As IDimXpertAnnotation  
    Dim feature As IDimXpertFeature
```

```

Dim appliedFeature As IDimXpertFeature
Dim msgStr As String
Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        ' Use IDimXpertFeature to get general feature data
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "    Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "    " + "Model feature: "
        ' Use IFeature to get the Swift model feature corresponding to this geometric dimensioning and tolerance feature
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeName2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "    " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "    " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
    
```

```

Debug.Print(msgStr + msgStr2)
appliedFeatures = feature.GetAppliedFeatures()
If Not (IsNothing(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        appliedFeature = appliedFeatures(o)
        Debug.Print("    " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If
msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print("    ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature
Dim cylinderFeature As IDimXpertCylinderFeature
cylinderFeature = swDXPart.GetFeature("Cylinder1")
msgStr = cylinderFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)

' Get the nominal top plane coordinates for Cylinder1
Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
boolstatus = cylinderFeature.GetNominalTopPlane(x, y, z, i, j, k)
msgStr = "X-coordinate of nominal top plane of " + cylinderFeature.Name + " is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)

' Get the thread for the cylinder, if it exists
Dim isThreaded As Boolean
Dim threadDesignation As String
Dim threadDepth As Double
boolstatus = cylinderFeature.GetThread(isThreaded, threadDesignation, threadDepth)
msgStr = "The cylinder is threaded? "
msgStr2 = isThreaded
Debug.Print(msgStr + msgStr2)
Debug.Print("")

' Get the radius of Fillet1
Dim filletFeature As IDimXpertFilletFeature
filletFeature = swDXPart.GetFeature("Fillet1")
msgStr = filletFeature.Name + " is a DimXpert feature"

```

```

Debug.Print("")
Debug.Print(msgStr)
Dim radius As Double
radius = filletFeature.radius
msgStr = "The fillet radius is "
msgStr2 = radius
Debug.Print(msgStr + msgStr2)
Dim planeFeature As IDimXpertPlaneFeature
planeFeature = swDXPart.GetFeature("Plane1")
msgStr = planeFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
' Get the nominal top plane coordinates for Plane1
boolstatus = planeFeature.GetNominalPlane(x, y, z, i, j, k)
msgStr = "X-coordinate of nominal top plane of " + planeFeature.Name + " is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
' Get the nominal closed slot width for Slot1
Dim slotFeature As IDimXpertCompoundClosedSlot3DFeature
slotFeature = swDXPart.GetFeature("Slot1")
msgStr = slotFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Dim width As Double
Dim length As Double
Dim longitudeI As Double
Dim longitudeJ As Double
Dim longitudeK As Double
boolstatus = slotFeature.GetNominalClosedSlot(width, length, x, y, z, i, j, k, longitudeI, longitudeJ, longitudeK)
msgStr = "Width of nominal closed slot is "
msgStr2 = width
Debug.Print(msgStr + msgStr2)
' Get the sub-features of Slot Pattern1
Dim patternFeature As IDimXpertPatternFeature
patternFeature = swDXPart.GetFeature("Slot Pattern1")
msgStr = patternFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
featureType = patternFeature.PatternType
Call GetPatternType(featureType, msgStr2)
Dim featureCount As Integer
featureCount = patternFeature.GetSubFeatureCount()
msgStr = "    Number of "
msgStr3 = featureCount
Debug.Print(msgStr + msgStr2 + " is " + msgStr3)
Dim subfeatures As Object
subfeatures = patternFeature.GetSubFeatures()
Debug.Print("    Sub-features of Slot Pattern1:")
For n = 0 To UBound(subfeatures)
    ' Use IDimXpertCompoundClosedSlot3DFeature to get the two planes of the slot
    ' The names of the planes are inherited from the Name property of the parent interface, IDimXpertFeature

```

```

    slotFeature = subfeatures(n)
    Debug.Print("    " + slotFeature.Name)
    Dim plane1 As IDimXpertPlaneFeature
    Dim plane2 As IDimXpertPlaneFeature
    boolstatus = slotFeature.GetPlaneFeatures(plane1, plane2)
    Debug.Print("    " + plane1.Name)
    Debug.Print("    " + plane2.Name)
Next
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Extrude) Then
        msgStr2 = "Boss"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_BestfitPlane) Then
        msgStr2 = "Bestfit plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeatureType_Surface) Then
        msgStr2 = "Surface"
    End If
End Sub

Public swApp As SldWorks
End Class

```

Get DimXpert Features and Annotations in a Model Example CSharp (Topic)

SOLIDWORKS API Help

Get DimXpert Features and Annotations in a Model Example (C#)

This application shows you how to get all of the DimXpert feature and annotation objects in a model.

This application is more advanced than other examples, because it involves multiple class modules.

```
//-----  
// Preconditions:  
// 1. Create a new C# macro using SOLIDWORKS VSTA.  
// 2. Name the project DimXpert_text_v2_cs.  
// 3. Save the project.  
// 4. Copy and paste the Main module into the code window.  
// 5. Right-click on the project in Project Explorer and click Add > Class.  
// 6. Select the Class template and type DimXpertFeatureData.cs  
//    in the Name field.  
// 7. Click Add.  
// 8. Copy and paste the DimXpertFeatureData class module into the code window.  
// 9. Right-click on the project in Project Explorer and click Add > Class.  
// 10. Select the Class template and type DimXpertAnnotationData.cs  
//    in the Name field.  
// 11. Click Add.  
// 12. Copy and paste the DimXpertAnnotationData class module  
//    into the code window.  
// 13. Ensure that the latest SolidWorks.Interop.swdimxpert.dll interop assembly  
//    is loaded (right-click on the project in Project Explorer,  
//    click Add Reference, choose the DimXpert assembly on the .NET tab).  
// 14. Ensure that the Microsoft Scripting Runtime library is loaded  
//    (right-click on the project in Project Explorer, click Add Reference,  
//    choose the library on the COM tab).  
// 15. Open a part that contains DimXpert features and/or annotations.  
//    (You may want to go through the online DimXpert Tutorial  
//    to learn how to create DimXpert features and annotations.  
//    From the ? help menu, click SOLIDWORKS Tutorials > All SOLIDWORKS  
//    Tutorials (Set 1) > DimXpert Tutorials)  
// 16. Open an Immediate Window in the IDE.  
// 17. Run this macro (F5).  
//  
// Postconditions:  
// 1. The output of this macro is logged in c:\temp\dimXpertInfo.txt.  
// 2. Inspect the Immediate Window.  
//-----
```

Get DimXpert Intersect Features Example (VBA)

This example shows how to build a part and get attributes for the following DimXpert features:

- * Intersect Point
- * Intersect Line
- * Intersect Circle
- * Intersect Plane

```
"-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilface_plate_ads_geo.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create an Intersect Point Dimxpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Create Intersection Point from the DimXpert shortcut menu.  
'   d. Select a cylinder whose axis intersects the first face at a point.  
'   e. Click the green check mark in the shortcut menu to create the  
'       intersect point.  
'   f. Select a feature on the part to dimension against the intersect point.  
'   g. Click to position the dimension in the view.  
'   h. Observe the new Dimxpert feature on the DimXpertManager tab:  
'       Intersect Point1  
' 4. Create an Intersect Line Dimxpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Create Intersection Line from the DimXpert shortcut menu.  
'   d. Select a perpendicular plane that would intersect the first plane  
'       if extended (e.g., a top or side face of the part).  
'   e. Click the green check mark in the shortcut menu to create the  
'       intersect line.  
'   f. Select a feature on the part to dimension against the intersect line.  
'   g. Click to position the dimension in the view.  
'   h. Observe the new Dimxpert feature on the DimXpertManager tab:  
'       Intersect Line1  
' 5. Create an Intersect Circle Dimxpert Feature:  
'   a. Click Size Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Intersect Circle from the DimXpert shortcut menu.  
'   d. Select the opening conical surface of a flat head machine screw  
'       (LPattern1 or LPattern3) in the part.  
'   e. Click the green check mark in the shortcut menu to finish the dimension.  
'   f. Click to position the size dimension in the view.  
'   g. Observe the new Dimxpert feature on the DimXpertManager tab:  
'       Intersect Circle1  
' 6. Create an Intersect Plane Dimxpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Zoom in on a flat head machine screw (LPattern1 or LPattern3).
```

- ' c. Select the opening conical surface of a flat head machine screw
- ' in the part.
- ' d. Select **Intersect Plane** from the DimXpert shortcut menu.
- ' e. Select the inner cylindrical bore face of the flat head machine screw.
- ' f. Click the green check mark in the pop-up menu to create the
- ' intersect plane.
- ' g. Select the top face of one of the extruded entities.
- ' h. Click to position the location dimension in the view.
- ' i. Observe the new Dimxpert feature on the DimXpertManager tab:
- ' **Intersect Plane1**
- ' 7. Open the Immediate window.
- ' 8. Ensure that the SOLIDWORKS DimXpert type library is loaded
- ' in **Tools > References**.
- '
- ' Postconditions: Compare the output in the Immediate Window with the features
- ' displayed on the DimXpertManager tab of the Management Panel.
- '
- ' NOTE: Because this part is used elsewhere, do not save changes.

'-----

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim swModel As ModelDoc2

Dim swModelDocExt As ModelDocExtension

Dim swSelMgr As SelectionMgr

Dim swConfig As Configuration

Dim swFeature As feature

Dim swAnn As feature

Dim swSchema As DimXpertManager

Dim swDXPart As DimXpertPart

Dim featureType As swDimXpertFeatureType_e

Dim features As Variant

Dim appliedFeatures As Variant

Dim appliedAnnotations As Variant

Dim appliedAnnotation As DimXpertAnnotation

Dim feature As DimXpertFeature

Dim feature1 As DimXpertFeature

Dim feature2 As DimXpertFeature

Dim appliedFeature As DimXpertFeature

Dim msgStr As String

Dim msgStr2 As String

Dim msgStr3 As String

Dim msgStr4 As String

Dim n As Long

Dim o As Long

Dim p As Long

Dim x As Double

Dim y As Double

Dim z As Double

Dim i As Double

Dim j As Double

Dim k As Double

```

Dim radius As Double
Dim boolstatus As Boolean
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Default", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = "   " + "Model feature: "
        Set swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetType
            Debug.Print msgStr + msgStr2
        End If

        msgStr = "   " + "Number of SOLIDWORKS face entities in this feature: "

```

```
msgStr2 = feature.GetFaceCount
```

```
Debug.Print msgStr + msgStr2
```

```
msgStr = "    " + "Number of applied features: "
```

```
msgStr2 = feature.GetAppliedFeatureCount()
```

```
Debug.Print msgStr + msgStr2
```

```
appliedFeatures = feature.GetAppliedFeatures()
```

```
If Not (IsEmpty(appliedFeatures)) Then
```

```
    For o = 0 To UBound(appliedFeatures)
```

```
        Set appliedFeature = appliedFeatures(o)
```

```
        Debug.Print "        " + "Applied feature name: " + (appliedFeature.Name)
```

```
    Next
```

```
End If
```

```
msgStr = "    " + "Number of applied annotations: "
```

```
msgStr2 = feature.GetAppliedAnnotationCount()
```

```
Debug.Print msgStr + msgStr2
```

```
appliedAnnotations = feature.GetAppliedAnnotations()
```

```
If Not (IsEmpty(appliedAnnotations)) Then
```

```
    For p = 0 To UBound(appliedAnnotations)
```

```
        Set appliedAnnotation = appliedAnnotations(p)
```

```
        Debug.Print "        " + "Applied annotation name: " + (appliedAnnotation.Name)
```

```
    Next
```

```
End If
```

```
Debug.Print "    "
```

```
Next
```

```
' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
```

```
' which can return a general IDimXpertFeature or a more specific interface on the feature
```

```
' Get IDimXpertIntersectCircleFeature for the Intersect Circle1 feature
```

```
Dim iCircleFeature As IDimXpertIntersectCircleFeature
```

```
Set iCircleFeature = swDXPart.GetFeature("Intersect Circle1")
```

```
msgStr = iCircleFeature.Name + " is a DimXpert intersect circle feature"
```

```
Debug.Print ""
```

```
Debug.Print msgStr
```

```
boolstatus = iCircleFeature.GetIntersectFeature(feature2)
```

```
Debug.Print "Intersect feature: " + feature2.Name
```

```
' Get the nominal circle parameters
```

```
boolstatus = iCircleFeature.GetNominalCircle(radius, x, y, z, i, j, k)
```

```
If boolstatus Then
```

```
    msgStr = "    The nominal circle has the following parameters:"
```

```
    Debug.Print msgStr
```

```
    msgStr = "        Radius is "
```

```
    msgStr2 = radius
```

```
    Debug.Print msgStr + msgStr2
```

```

msgStr = "    x-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "    y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2
msgStr = "    z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
msgStr = "    i-component of pierce vector is "
msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "    j-component of pierce vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "    k-component of pierce vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2
End If

```

```

Dim plane As IDimXpertPlaneFeature
boolstatus = iCircleFeature.GetPlaneFeature(plane)
If boolstatus Then
    msgStr = "    The nominal circle has the following plane feature: "
    Debug.Print msgStr
    msgStr2 = plane.Name
    Debug.Print msgStr + msgStr2
End If

```

```

' Get IDimXpertIntersectLineFeature for the Intersect Line1 feature
Dim iLineFeature As IDimXpertIntersectLineFeature
Set iLineFeature = swDXPart.GetFeature("Intersect Line1")
msgStr = iLineFeature.Name + " is a DimXpert intersect line feature"
Debug.Print ""
Debug.Print msgStr

```

```

' Get the nominal line parameters
boolstatus = iLineFeature.GetNominalLine(x, y, z, i, j, k)
If boolstatus Then
    msgStr = "    The nominal line has the following parameters:"
    Debug.Print msgStr
    msgStr = "    x-coordinate is "
    msgStr2 = x
    Debug.Print msgStr + msgStr2
    msgStr = "    y-coordinate is "
    msgStr2 = y
    Debug.Print msgStr + msgStr2
    msgStr = "    z-coordinate is "
    msgStr2 = z
    Debug.Print msgStr + msgStr2
    msgStr = "    i-component of pierce vector is "

```

```

msgStr2 = i
Debug.Print msgStr + msgStr2
msgStr = "    j-component of pierce vector is "
msgStr2 = j
Debug.Print msgStr + msgStr2
msgStr = "    k-component of pierce vector is "
msgStr2 = k
Debug.Print msgStr + msgStr2

```

End If

```

    Dim plane1 As IDimXpertPlaneFeature
    Dim plane2 As IDimXpertPlaneFeature

```

```

boolstatus = iLineFeature.GetPlaneFeatures(plane1, plane2)

```

If boolstatus Then

```

    msgStr = "    The nominal line has the following plane features:"
    Debug.Print msgStr
    msgStr = "        plane1 is "
    msgStr2 = plane1.Name
    Debug.Print msgStr + msgStr2
    msgStr = "        plane2 is "
    msgStr2 = plane2.Name
    Debug.Print msgStr + msgStr2

```

End If

```

' Get IDimXpertIntersectPlaneFeature for the Intersect Plane1 feature
Dim iPlaneFeature As IDimXpertIntersectPlaneFeature
Set iPlaneFeature = swDXPart.GetFeature("Intersect Plane1")
msgStr = iPlaneFeature.Name + " is a DimXpert intersect plane feature"
Debug.Print ""
Debug.Print msgStr

```

```

' Get the two features used to construct the intersect plane
boolstatus = iPlaneFeature.GetFeatures(feature1, feature2)

```

```

    msgStr = "    The plane intersects the following DimXpert features:"
    Debug.Print msgStr
    msgStr = "        Feature1 is "
    msgStr2 = feature1.Name
    Debug.Print msgStr + msgStr2
    msgStr = "        Feature2 is "
    msgStr2 = feature2.Name
    Debug.Print msgStr + msgStr2

```

```

' Get IDimXpertIntersectPointFeature for the Intersect Point1 feature
Dim iPointFeature As IDimXpertIntersectPointFeature
Set iPointFeature = swDXPart.GetFeature("Intersect Point1")
msgStr = iPointFeature.Name + " is a DimXpert intersect point feature"
Debug.Print ""

```

```
Debug.Print msgStr
```

```
' Get the nominal point
```

```
boolstatus = iPointFeature.GetNominalPoint(x, y, z)
```

```
If boolstatus Then
```

```
    msgStr = "  The nominal point has the following parameters:"
```

```
    Debug.Print msgStr
```

```
    msgStr = "    x-coordinate is "
```

```
    msgStr2 = x
```

```
    Debug.Print msgStr + msgStr2
```

```
    msgStr = "    y-coordinate is "
```

```
    msgStr2 = y
```

```
    Debug.Print msgStr + msgStr2
```

```
    msgStr = "    z-coordinate is "
```

```
    msgStr2 = z
```

```
    Debug.Print msgStr + msgStr2
```

```
End If
```

```
End Sub
```

```
Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
```

```
    If (featureType = swDimXpertFeature_Plane) Then
```

```
        msgStr2 = "Plane"
```

```
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
```

```
        msgStr2 = "Cylinder"
```

```
    ElseIf (featureType = swDimXpertFeature_Cone) Then
```

```
        msgStr2 = "Cone"
```

```
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
```

```
        msgStr2 = "Extrude"
```

```
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
```

```
        msgStr2 = "Fillet"
```

```
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
```

```
        msgStr2 = "Chamfer"
```

```
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
```

```
        msgStr2 = "CompoundHole"
```

```
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
```

```
        msgStr2 = "CompoundWidth"
```

```
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
```

```
        msgStr2 = "CompoundNotch"
```

```
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
```

```
        msgStr2 = "CompoundClosedSlot3D"
```

```
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
```

```
        msgStr2 = "IntersectPoint"
```

```
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
```

```
        msgStr2 = "IntersectLine"
```

```
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
```

```
        msgStr2 = "IntersectCircle"
```

```
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
```

```
        msgStr2 = "IntersectPlane"
```

```
    ElseIf (featureType = swDimXpertFeature_Pattern) Then
```

```
        msgStr2 = "Pattern"
```

```
    ElseIf (featureType = swDimXpertFeature_Sphere) Then
```

```
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub---
```

Get DimXpert Intersect Features Example (VB.NET)

This example shows how to build a part and get attributes for the following DimXpert features:

- * Intersect Point
- * Intersect Line
- * Intersect Circle
- * Intersect Plane

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\apilface_plate_ads_geo.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Create an Intersect Point DimXpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Create Intersection Point on the DimXpert shortcut menu.  
'   d. Select a cylinder whose axis intersects the first face at a point.  
'   e. Click the green check mark in the shortcut menu to create the  
'       intersect point.  
'   f. Select a feature on the part to dimension against the intersect point.  
'   g. Click to position the dimension in the view.  
'   h. Observe the new DimXpert feature on the DimXpertManager tab:  
'       Intersect Point1  
' 4. Create an Intersect Line DimXpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Create Intersection Line from the DimXpert shortcut menu.  
'   d. Select a perpendicular plane that would intersect the first plane  
'       if extended (e.g., a top or side face of the part).  
'   e. Click the green check mark in the shortcut menu to create the  
'       intersect line.  
'   f. Select a feature on the part to dimension against the intersect line.  
'   g. Click to position the dimension in the view.  
'   h. Observe the new DimXpert feature on the DimXpertManager tab:  
'       Intersect Line1  
' 5. Create an Intersect Circle DimXpert Feature:  
'   a. Click Size Dimension on the DimXpert toolbar.  
'   b. Select the front face of the part.  
'   c. Select Intersect Circle from the DimXpert shortcut menu.  
'   d. Select the opening conical surface of a flat head machine screw  
'       (LPattern1 or LPattern3) in the part.  
'   e. Click the green check mark in the shortcut menu to finish the dimension.  
'   f. Click to position the size dimension in the view.  
'   g. Observe the new DimXpert feature on the DimXpertManager tab:  
'       Intersect Circle1  
' 6. Create an Intersect Plane DimXpert Feature:  
'   a. Click Location Dimension on the DimXpert toolbar.  
'   b. Zoom in on a flat head machine screw (LPattern1 or LPattern3).
```

- ' c. Select the opening conical surface of a flat head machine screw
- ' in the part.
- ' d. Select **Intersect Plane** from the DimXpert shortcut menu.
- ' e. Select the inner cylindrical bore face of the flat head machine screw.
- ' f. Click the green check mark in the shortcut menu to create the
- ' intersect plane.
- ' g. Select the top face of one of the extruded entities.
- ' h. Click to position the location dimension in the view.
- ' i. Observe the new DimXpert feature on the DimXpertManager tab:
- ' **Intersect Plane1**
- ' 7. Open an Immediate window.
- ' 8. Ensure that the latest **SolidWorks.Interop.swdimxpert.dll** interop
- ' assembly is loaded (right-click project in Project Explorer and
- ' click **Add Reference** > **.NET** tab).
- '
- ' Postconditions: Compare the output in the Immediate Window
- ' with the features displayed on the DimXpertManager tab of the Management Panel.
- '
- ' NOTE: Because this part is used elsewhere, do not save changes.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swdimxpert
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro
```

```
Dim swModel As IModelDoc2
Dim swModelDocExt As IModelDocExtension
Dim swSelMgr As ISelectionMgr
Dim swConfig As IConfiguration
Dim swFeature As IFeature
Dim swAnn As IFeature
Dim swSchema As IDimXpertManager
Dim swDXPart As IDimXpertPart
Dim featureType As swDimXpertFeatureType_e
Dim features As Object
Dim appliedFeatures As Object
Dim appliedAnnotations As Object
Dim appliedAnnotation As IDimXpertAnnotation
Dim feature As IDimXpertFeature
Dim feature1 As IDimXpertFeature
Dim feature2 As IDimXpertFeature
Dim appliedFeature As IDimXpertFeature
Dim msgStr As String
Dim msgStr2 As String
Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
```

```

Dim x As Double
Dim y As Double
Dim z As Double
Dim i As Double
Dim j As Double
Dim k As Double
Dim radius As Double
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Default", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " DimXpert features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "   " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "   " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "   " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then

```

```

    For o = 0 To UBound(appliedFeatures)
        appliedFeature = appliedFeatures(o)
        Debug.Print("    " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If
msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print("    ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

' Get IDimXpertIntersectCircleFeature for the Intersect Circle1 feature
Dim iCircleFeature As IDimXpertIntersectCircleFeature
iCircleFeature = swDXPart.GetFeature("Intersect Circle1")
msgStr = iCircleFeature.Name + " is a DimXpert intersect circle feature"
Debug.Print("")
Debug.Print(msgStr)
' Get the nominal circle parameters
boolstatus = iCircleFeature.GetNominalCircle(radius, x, y, z, i, j, k)
If boolstatus Then
    msgStr = "    The nominal circle has the following parameters:"
    Debug.Print(msgStr)
    msgStr = "    Radius is "
    msgStr2 = radius
    Debug.Print(msgStr + msgStr2)
    msgStr = "    x-coordinate is "
    msgStr2 = x
    Debug.Print(msgStr + msgStr2)
    msgStr = "    y-coordinate is "
    msgStr2 = y
    Debug.Print(msgStr + msgStr2)
    msgStr = "    z-coordinate is "
    msgStr2 = z
    Debug.Print(msgStr + msgStr2)
    msgStr = "    i-component of pierce vector is "
    msgStr2 = i
    Debug.Print(msgStr + msgStr2)
    msgStr = "    j-component of pierce vector is "
    msgStr2 = j
    Debug.Print(msgStr + msgStr2)
    msgStr = "    k-component of pierce vector is "
    msgStr2 = k

```

```

    Debug.Print(msgStr + msgStr2)
End If
' Get IDimXpertIntersectLineFeature for the Intersect Line1 feature
Dim iLineFeature As IDimXpertIntersectLineFeature
iLineFeature = swDXPart.GetFeature("Intersect Line1")
msgStr = iLineFeature.Name + " is a DimXpert intersect line feature"
Debug.Print("")
Debug.Print(msgStr)
' Get the nominal line parameters
boolstatus = iLineFeature.GetNominalLine(x, y, z, i, j, k)
If boolstatus Then
    msgStr = " The nominal line has the following parameters:"
    Debug.Print(msgStr)
    msgStr = " x-coordinate is "
    msgStr2 = x
    Debug.Print(msgStr + msgStr2)
    msgStr = " y-coordinate is "
    msgStr2 = y
    Debug.Print(msgStr + msgStr2)
    msgStr = " z-coordinate is "
    msgStr2 = z
    Debug.Print(msgStr + msgStr2)
    msgStr = " i-component of pierce vector is "
    msgStr2 = i
    Debug.Print(msgStr + msgStr2)
    msgStr = " j-component of pierce vector is "
    msgStr2 = j
    Debug.Print(msgStr + msgStr2)
    msgStr = " k-component of pierce vector is "
    msgStr2 = k
    Debug.Print(msgStr + msgStr2)
End If
' Get IDimXpertIntersectPlaneFeature for the Intersect Plane1 feature
Dim iPlaneFeature As IDimXpertIntersectPlaneFeature
iPlaneFeature = swDXPart.GetFeature("Intersect Plane1")
msgStr = iPlaneFeature.Name + " is a DimXpert intersect plane feature"
Debug.Print("")
Debug.Print(msgStr)
' Get the two features used to construct the intersect plane
boolstatus = iPlaneFeature.GetFeatures(feature1, feature2)
msgStr = " The plane intersects the following DimXpert features:"
Debug.Print(msgStr)
msgStr = " Feature1 is "
msgStr2 = feature1.Name
Debug.Print(msgStr + msgStr2)
msgStr = " Feature2 is "
msgStr2 = feature2.Name
Debug.Print(msgStr + msgStr2)
' Get IDimXpertIntersectPointFeature for the Intersect Point1 feature
Dim iPointFeature As IDimXpertIntersectPointFeature
iPointFeature = swDXPart.GetFeature("Intersect Point1")

```

```

msgStr = iPointFeature.Name + " is a DimXpert intersect point feature"
Debug.Print("")
Debug.Print(msgStr)
' Get the nominal point
boolstatus = iPointFeature.GetNominalPoint(x, y, z)
If boolstatus Then
    msgStr = " The nominal point has the following parameters:"
    Debug.Print(msgStr)
    msgStr = " x-coordinate is "
    msgStr2 = x
    Debug.Print(msgStr + msgStr2)
    msgStr = " y-coordinate is "
    msgStr2 = y
    Debug.Print(msgStr + msgStr2)
    msgStr = " z-coordinate is "
    msgStr2 = z
    Debug.Print(msgStr + msgStr2)
End If
End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then

```

```
    msgStr2 = "Sphere"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
Elseif (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If
End Sub

Public swApp As SldWorks
End Class
```

Get_DimXpert_Sphere_Feature_Example_VB (Topic)

SOLIDWORKS API Help

Get DimXpert Sphere Feature Example (VBA)

This example shows how to build a part and get attributes for the DimXpert Sphere feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosflopress\ball valve\ball.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click the Datum on the DimXpert toolbar.  
' 4. Click the ball of the part.  
' 5. Click to place the datum annotation.  
' 6. Click the green check mark to accept the new datum feature.  
' 7. Observe the following DimXpert features on the DimXpertManager tab:  
'   Sphere1.  
' 8. Open an Immediate window  
' 9. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'   in Tools > References.  
'  
' Postconditions: Compare the output in the Immediate Window  
' with the features displayed on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

Dim swApp As SldWorks.SldWorks

Dim swModel As ModelDoc2

Dim swModelDocExt As ModelDocExtension

Dim swSelMgr As SelectionMgr

Dim swConfig As Configuration

Dim swFeature As feature

Dim swAnn As feature

Dim swSchema As DimXpertManager

Dim swDXPart As DimXpertPart

Dim featureType As swDimXpertFeatureType_e

Dim features As Variant

Dim appliedFeatures As Variant

Dim appliedAnnotations As Variant

Dim appliedAnnotation As DimXpertAnnotation

Dim feature As DimXpertFeature

Dim appliedFeature As DimXpertFeature

Dim msgStr As String

Dim msgStr2 As String

Dim msgStr3 As String

Dim msgStr4 As String

Dim n As Long

Dim o As Long

Dim p As Long

Dim boolstatus As Boolean

```

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()

    Set swSchema = swModelDocExt.DimXpertManager("Standard", True)

    ' Get IDimXpertPart from the IDimXpertManager
    Set swDXPart = swSchema.DimXpertPart

    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print ""
    Debug.Print msgStr

    ' Get IDimXpert features through IDimXpertPart

    features = swDXPart.GetFeatures

    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print ""
    Debug.Print msgStr
    For n = 0 To UBound(features)
        Set feature = features(n)
        Debug.Print " " + "Feature name: " + (feature.Name)

        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)

        msgStr = " " + "Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print msgStr + msgStr2 + msgStr3 + msgStr4

        msgStr = " " + "Model feature: "
        Set swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print msgStr + msgStr2
        End If

        msgStr = " " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print msgStr + msgStr2
    
```

```

msgStr = "    " + "Number of applied features: "
msgStr2 = feature.GetAppliedFeatureCount()
Debug.Print msgStr + msgStr2

```

```

appliedFeatures = feature.GetAppliedFeatures()
If Not (IsEmpty(appliedFeatures)) Then
    For o = 0 To UBound(appliedFeatures)
        Set appliedFeature = appliedFeatures(o)
        Debug.Print "    " + "Applied feature name: " + (appliedFeature.Name)
    Next
End If

```

```

msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print msgStr + msgStr2

```

```

appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsEmpty(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        Set appliedAnnotation = appliedAnnotations(p)
        Debug.Print "    " + "Applied annotation name: " + (appliedAnnotation.Name)
    Next
End If

```

```

Debug.Print "    "
Next

```

```

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

```

```

' Get IDimXpertSphereFeature for the Sphere-101 feature

```

```

Dim sphereFeature As IDimXpertSphereFeature
Set sphereFeature = swDXPart.GetFeature("Sphere-101")
msgStr = sphereFeature.Name + " is a DimXpert feature"
Debug.Print ""
Debug.Print msgStr
Debug.Print ""

```

```

' Get the nominal sphere coordinates

```

```

Dim radius As Double
Dim x As Double
Dim y As Double
Dim z As Double

```

```

Debug.Print "Nominal sphere of Sphere-101"
Debug.Print ""
boolstatus = sphereFeature.GetNominalSphere(radius, x, y, z)
msgStr = "Radius is "

```

```

msgStr2 = radius
Debug.Print msgStr + msgStr2
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print msgStr + msgStr2
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print msgStr + msgStr2
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print msgStr + msgStr2
Debug.Print ""

```

' Get whether the sphere is a hole or a pin

```

boolstatus = sphereFeature.Inner
msgStr = "The sphere is a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print msgStr + msgStr2

```

End Sub

```

Public Sub GetPatternType(ByRef featureType, ByRef msgStr2)
    If (featureType = swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeature_Pattern) Then

```

```
    msgStr2 = "Pattern"
ElseIf (featureType = swDimXpertFeature_Sphere) Then
    msgStr2 = "Sphere"
ElseIf (featureType = swDimXpertFeature_BestfitPlane) Then
    msgStr2 = "Bestfit Plane"
ElseIf (featureType = swDimXpertFeature_Surface) Then
    msgStr2 = "Surface"
End If

End Sub
```

Get DimXpert Sphere Feature Example (VB.NET)

This example shows how to build a part and get attributes for the DimXpert Sphere feature.

```
'-----  
' Preconditions:  
' 1. Open public_documents\tutorial\cosmosflopressball_valve\ball.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
' 3. Click Datum on the DimXpert toolbar.  
' 4. Click the ball of the part.  
' 5. Click to place the datum annotation.  
' 6. Click the green check mark to accept the new datum feature.  
' 7. Observe the following DimXpert features on the DimXpertManager tab:  
'   Sphere1.  
' 8. Open an Immediate window.  
' 9. Ensure that the SolidWorks.Interop.swdimxpert.dll interop assembly  
'    is loaded (right-click project in Project Explorer and click Add Reference >  
'    .NET tab).  
'  
' Postconditions: Compare the output in the Immediate Window with the features displayed  
' on the DimXpertManager tab of the Management Panel.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Dim swModel As IModelDoc2  
Dim swModelDocExt As IModelDocExtension  
Dim swSelMgr As ISelectionMgr  
Dim swConfig As IConfiguration  
Dim swFeature As IFeature  
Dim swAnn As IFeature  
Dim swSchema As IDimXpertManager  
Dim swDXPart As IDimXpertPart  
Dim featureType As swDimXpertFeatureType_e  
Dim features As Object  
Dim appliedFeatures As Object  
Dim appliedAnnotations As Object  
Dim appliedAnnotation As IDimXpertAnnotation  
Dim feature As IDimXpertFeature  
Dim appliedFeature As IDimXpertFeature  
Dim msgStr As String  
Dim msgStr2 As String
```

```

Dim msgStr3 As String
Dim msgStr4 As String
Dim n As Long
Dim o As Long
Dim p As Long
Dim boolstatus As Boolean
Sub main()
    swModel = swApp.ActiveDoc
    swModelDocExt = swModel.Extension
    swSelMgr = swModel.SelectionManager
    ' Get the default DimXpert schema using IModelDocExtension.DimXpertManager()
    swSchema = swModelDocExt.DimXpertManager("Standard", True)
    ' Get IDimXpertPart from the IDimXpertManager
    swDXPart = swSchema.DimXpertPart
    Dim featCount As Long
    featCount = swDXPart.GetFeatureCount
    msgStr = "Total of "
    msgStr2 = featCount
    msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
    Debug.Print("")
    Debug.Print(msgStr)
    ' Get IDimXpert features through IDimXpertPart
    features = swDXPart.GetFeatures
    msgStr = (swSchema.SchemaName) + " has the following features: "
    Debug.Print("")
    Debug.Print(msgStr)
    For n = 0 To UBound(features)
        feature = features(n)
        Debug.Print(" " + "Feature name: " + (feature.Name))
        featureType = feature.Type
        Call GetPatternType(featureType, msgStr2)
        msgStr = "   Feature type "
        msgStr3 = " is suppressed on the DimXpertManager tab? "
        msgStr4 = feature.IsSuppressed()
        Debug.Print(msgStr + msgStr2 + msgStr3 + msgStr4)
        msgStr = "   " + "Model feature: "
        swFeature = feature.GetModelFeature()
        If Not (swFeature Is Nothing) Then
            msgStr2 = swFeature.GetTypeNames2()
            Debug.Print(msgStr + msgStr2)
        End If
        msgStr = "   " + "Number of SOLIDWORKS face entities in this feature: "
        msgStr2 = feature.GetFaceCount
        Debug.Print(msgStr + msgStr2)
        msgStr = "   " + "Number of applied features: "
        msgStr2 = feature.GetAppliedFeatureCount()
        Debug.Print(msgStr + msgStr2)
        appliedFeatures = feature.GetAppliedFeatures()
        If Not (IsNothing(appliedFeatures)) Then
            For o = 0 To UBound(appliedFeatures)
                appliedFeature = appliedFeatures(o)
            
```

```

        Debug.Print("    " + "Applied feature name: " + (appliedFeature.Name))
    Next
End If
msgStr = "    " + "Number of applied annotations: "
msgStr2 = feature.GetAppliedAnnotationCount()
Debug.Print(msgStr + msgStr2)
appliedAnnotations = feature.GetAppliedAnnotations()
If Not (IsNothing(appliedAnnotations)) Then
    For p = 0 To UBound(appliedAnnotations)
        appliedAnnotation = appliedAnnotations(p)
        Debug.Print("    " + "Applied annotation name: " + (appliedAnnotation.Name))
    Next
End If
Debug.Print(" ")
Next

' If you know the name of a DimXpert feature, you can get it directly using IDimXpertPart.GetFeature("name"),
' which can return a general IDimXpertFeature or a more specific interface on the feature

' Get IDimXpertSphereFeature for the Sphere-101 feature
Dim sphereFeature As IDimXpertSphereFeature
sphereFeature = swDXPart.GetFeature("Sphere-101")
msgStr = sphereFeature.Name + " is a DimXpert feature"
Debug.Print("")
Debug.Print(msgStr)
Debug.Print("")
' Get the nominal sphere coordinates
Dim radius As Double
Dim x As Double
Dim y As Double
Dim z As Double
Debug.Print("Nominal sphere of Sphere-101")
Debug.Print("")
boolstatus = sphereFeature.GetNominalSphere(radius, x, y, z)
msgStr = "Radius is "
msgStr2 = radius
Debug.Print(msgStr + msgStr2)
msgStr = "X-coordinate is "
msgStr2 = x
Debug.Print(msgStr + msgStr2)
msgStr = "Y-coordinate is "
msgStr2 = y
Debug.Print(msgStr + msgStr2)
msgStr = "Z-coordinate is "
msgStr2 = z
Debug.Print(msgStr + msgStr2)
Debug.Print("")
' Get whether the sphere is a hole or a pin
boolstatus = sphereFeature.Inner
msgStr = "The sphere is a hole and not a pin: "
msgStr2 = boolstatus
Debug.Print(msgStr + msgStr2)

```

```

End Sub

Public Sub GetPatternType(ByRef featureType As swDimXpertFeatureType_e, ByRef msgStr2 As String)
    If (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Plane) Then
        msgStr2 = "Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cylinder) Then
        msgStr2 = "Cylinder"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Cone) Then
        msgStr2 = "Cone"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Extrude) Then
        msgStr2 = "Extrude"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Fillet) Then
        msgStr2 = "Fillet"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Chamfer) Then
        msgStr2 = "Chamfer"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundHole) Then
        msgStr2 = "CompoundHole"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundWidth) Then
        msgStr2 = "CompoundWidth"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundNotch) Then
        msgStr2 = "CompoundNotch"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_CompoundClosedSlot3D) Then
        msgStr2 = "CompoundClosedSlot3D"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPoint) Then
        msgStr2 = "IntersectPoint"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectLine) Then
        msgStr2 = "IntersectLine"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectCircle) Then
        msgStr2 = "IntersectCircle"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_IntersectPlane) Then
        msgStr2 = "IntersectPlane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Pattern) Then
        msgStr2 = "Pattern"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Sphere) Then
        msgStr2 = "Sphere"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_BestfitPlane) Then
        msgStr2 = "Bestfit Plane"
    ElseIf (featureType = swDimXpertFeatureType_e.swDimXpertFeature_Surface) Then
        msgStr2 = "Surface"
    End If
End Sub

```

```

Public swApp As SldWorks
End Class

```